CMIS 3153 · ADVANCED DATABASE SYSTEMS

Mid Exam Study Pack · Lesson 3

Query Processing and Relational Algebra

Lesson in simple English · Formula cheat box · Answer recipes · Tutorial answers · Past paper questions 2020/21 – 2023/24 with answers

INSIDE THIS PACK

- How to use this pack
- SQL basics you must know before relational algebra
- Lesson part 1: How a database processes a query
- Lesson part 2: Relational algebra operations
- Cheat box: relational algebra at a glance
- How-to-answer recipes
- Tutorial 3: Relational model and relational algebra (SQL + RA)
- Tutorial 5: Relational algebra practice (SQL + RA)
- Lesson exercise (from the slides)
- Past paper questions with answers

★ asked in more than one past paper · (25m) marks in the paper · Yellow boxes = past paper questions · Blue boxes = formulas and key notes · Green boxes = given data

Contents

How to use this pack 5

Quick-find table 5

SQL basics you must know before relational algebra 7

0.1 The sample tables used in this section 7

0.2 SELECT ... FROM: choose columns (RA: π project) 7

0.3 WHERE: choose rows (RA: σ select) 8

0.4 Combining both 8

0.5 Joining two tables (RA: ⋈ join) 8

0.6 Subqueries: IN and NOT IN (RA: join and difference) 9

0.7 UNION, INTERSECT, EXCEPT (RA: ∪, ∩, −) 10

0.8 Aggregates, GROUP BY, HAVING (RA: γ grouping) 10

0.9 ORDER BY 10

0.10 The most important rule: the order SQL really runs in 10

Lesson part 1: How a database processes a query 12

1.1 The stages, in order 12

1.2 Why relational algebra matters for query processing 13

1.3 Pipelining vs materialization 13

Lesson part 2: Relational algebra operations 14

2.1 The basic operations 14

2.2 Joins 14

2.3 Division ÷ (for "all" / "every" questions) 15

2.4 Aggregation γ (grouping) 15

2.5 NULL values and multiset (bag) relational algebra 15

2.6 Comparing operations (lecture exercise) 16

2.7 Operator precedence and associativity 16

2.8 RA and database design (lecture exercise) 16

Cheat box: relational algebra at a glance 17

Symbol table 17

SQL clause → RA (build from the inside out) 17

English wording → operator 18

Key facts to remember 18

How-to-answer recipes 19

Recipe 1: English → RA with Filter → Join → Reduce ★ 19

Recipe 2: "not", "never", "does not" → difference ★ 19

Recipe 3: "all" / "every" → division 19

Recipe 4: "at least two different" → self-join 20

Recipe 5: SQL → RA (and checking a given translation) 20

Recipe 6: "Describe what this RA returns" and "compare two expressions" 20

Recipe 7: "Steps in evaluating an SQL query" ★ 20

Tutorial 3: Relational model and relational algebra (SQL + RA) 21

Q1. What is a relation, and what two properties must its elements have? 21

Q2. Relation vs relation schema 21

Q3. r1 over N1({A, B}, {A}) and r2 over N2({B, C}, {B}). Express N1[B] ⊆ N2[B] using projection. 21

Q4. RA queries 21

Tutorial 5: Relational algebra practice (SQL + RA) 24

(i) List all student names. 24

(ii) Student IDs of students in the "IT" department. 24

(iii) Course IDs of courses taught by faculty from the "CSE" department. 24

(iv) Names of students enrolled in at least one course. 24

(v) Course titles taken by students from the "ECE" department. 25

(vi) Names of faculty who do not teach any course. 25

(vii) Names of students who received grade 'A' in courses offered by the "CSE" department. 25

(viii) Names of students enrolled in all courses offered by the "IT" department. 25

(ix) Names of students enrolled in at least two different courses. 26

(x) Student names who have never received grade 'F'. 26

(xi) Course titles not taken by any student. 27

(xii) Names of faculty who teach only CSE department courses. 27

(xiii) Student names enrolled in every course taught by faculty from the ECE department. 27

Pattern summary 28

Lesson exercise (from the slides) 29

Exercise 1. Employee(EmpID, Name, Age, DeptID), Department(DeptID, DeptName), Project(ProjectID, DeptID, ProjectName), Assignment(EmpID, ProjectID). Find the names of employees who work on a project in the "IT" department and are older than 30. 29

Past paper questions with answers 30

A. Query processing steps 30
B. Translating SQL into relational algebra 31
C. Relational algebra theory 32
D. Writing relational algebra queries ★ 33
Questions NOT in the mid scope 37

How to use this pack

This lesson has come up in **every past paper**, usually as **Question 4**: "steps in evaluating a query" plus a set of **relational algebra (RA) queries** to write. The RA queries look hard, but they follow a few fixed patterns, and **Tutorial 3 Q4 came in the 2020/21 paper word for word**.

The order to study this pack:

1. **SQL basics** (next section). RA is just SQL written with symbols, so if you can read the SQL, you can write the RA.
2. The **symbol table** in the Cheat box: keep that page open in the exam.
3. The **recipes**: Filter → Join → Reduce, and the patterns for "not", "all", "at least two" and "only".
4. Tutorials 3 and 5: every query is given in **SQL and RA side by side**.

In the exam hall: find the wording in the Quick-find table, copy the matching pattern, change the table and column names.

Signs used in this pack: ★ = asked in more than one past paper. (25m) = marks in the paper. σ = select (filter rows), π = project (keep columns), ⋈ = join.

Quick-find table

If the question asks about...	Go to
Steps in evaluating an SQL query submitted as a string ★	Lesson part 1 · 2020/21 Q1(c), 2022/23 Q4(a), 2023/24 Q4(a)
Query processing stages and techniques for efficiency	Lesson part 1 · 2023/24 Q5(a)
Pipelining vs materialization	Lesson part 1
What a relation is; relation vs relation schema	Tutorial 3 Q1–Q2
Foreign key / referential integrity in RA	Tutorial 3 Q3
Write RA for English queries (works for a company, lives in a city...) ★	Recipes · Tutorial 3 Q4 = 2020/21 Q7(d) · 2022/23 Q4(c)
Write RA with only σ, π, × and natural join (Suppliers/Parts)	2021/22 Q5
Sailors/Boats/Reserves RA queries	2023/24 Q4(c)
"Not", "never", "does not" queries	Recipe 2 (difference) · Tutorial 5 (vi), (x), (xi)
"All" / "every" queries	Recipe 3 (division) · Tutorial 5 (viii), (xiii)
"At least two different" queries	Recipe 4 (self-join) · Tutorial 5 (ix)
"Describe what this RA expression returns, compare two"	Recipe 6 · 2022/23 Q4(c)(ii), 2023/24 Q4(c)(ii)
Is an RA translation of SQL (with GROUP BY / HAVING) correct?	Recipe 5 · 2020/21 Q1(a)

If the question asks about...	Go to
SQL with NOT IN → RA / expression tree	2020/21 Q1(b)
Operator precedence and associativity	Lesson part 2 · 2020/21 Q7(b)
Is π(E1 − E2) = π(E1) − π(E2)?	2020/21 Q7(c)
Can a key of one table be a key of another?	2021/22 Q5(a)

SQL basics you must know before relational algebra

Relational algebra (RA) is **SQL written with maths symbols**. Every SQL keyword has an RA symbol. Learn the SQL here first, then the RA in the next sections will feel familiar.

0.1 The sample tables used in this section

These are the Tutorial 5 tables with a few rows of made-up data.

STUDENT(SID, Name, Dept, Age):

SID	Name	Dept	Age
1	Amal	IT	21
2	Binu	CSE	23
3	Chamal	IT	22

COURSE(CID, Title, Dept):

CID	Title	Dept
C1	Databases	IT
C2	Networks	CSE
C3	AI	IT

ENROLL(SID, CID, Grade):

SID	CID	Grade
1	C1	A
1	C2	B
2	C1	F

- **Table = relation. Row = tuple. Column = attribute.**
- **Primary key** = the column that identifies each row uniquely (SID in STUDENT, CID in COURSE, (SID, CID) together in ENROLL).
- **Foreign key** = a column that points to a key in another table (ENROLL.SID points to STUDENT.SID). Foreign keys are what you **join on**.

0.2 SELECT ... FROM: choose columns (RA: π project)

```
SELECT Name, Dept
FROM STUDENT;
```

- `SELECT Name, Dept` = which **columns** to show. `SELECT *` = all columns.

- `FROM STUDENT` = which **table**.
- Result: (Amal, IT), (Binu, CSE), (Chamal, IT).
- `SELECT DISTINCT Dept FROM STUDENT;` removes duplicates → IT, CSE. (RA always removes duplicates, so π behaves like SELECT DISTINCT.)
- **RA:** π Name, Dept (STUDENT)

0.3 WHERE: choose rows (RA: σ select)

```
SELECT *
FROM STUDENT
WHERE Dept = 'IT' AND Age > 21;
```

- `WHERE` keeps only the rows where the condition is **true**. Result: (3, Chamal, IT, 22).
- Conditions: `=`, `<>` (not equal), `<`, `>`, `<=`, `>=`, `AND`, `OR`, `NOT`.
- Useful extras: `Age BETWEEN 20 AND 22` (from 20 to 22), `Dept IN ('IT', 'CSE')` (one of a list), `Name LIKE 'A%'` (starts with A).
- Text values go in **single quotes**: `'IT'`. Numbers do not: `21`.
- **RA:** σ Dept='IT' ∧ Age>21 (STUDENT). (∧ means AND, ∨ means OR, ¬ means NOT.)

Careful with names: SQL's SELECT = RA's **π** (columns). RA's **σ** ("select") = SQL's **WHERE** (rows).

0.4 Combining both

```
SELECT Name
FROM STUDENT
WHERE Dept = 'IT';
```

- Result: Amal, Chamal.
- **RA:** π Name (σ Dept='IT' (STUDENT)). Read RA **from the inside out**: first filter rows (σ), then keep columns (π).

0.5 Joining two tables (RA: ⋈ join)

To use data from two tables, match rows on the **linking column** (the foreign key).

```
SELECT S.Name, E.CID, E.Grade
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID;
```

- `STUDENT S` gives the table a short nickname (**alias**) S. Then `S.Name` means "Name from STUDENT".
- `JOIN ENROLL E ON S.SID = E.SID` = put each student row next to **their** enrolment rows (same SID).
- Result: (Amal, C1, A), (Amal, C2, B), (Binu, C1, F). Chamal has no enrolment, so Chamal is **not** in the result.

- The **old style** does the same thing: `FROM STUDENT S, ENROLL E WHERE S.SID = E.SID`. The comma = Cartesian product (×), and the WHERE condition keeps only matching pairs.
- **RA:** π Name, CID, Grade (STUDENT ⋈ ENROLL). The **natural join ⋈** automatically matches columns with the **same name** (SID here).

Three tables: add another JOIN.

```
SELECT S.Name, C.Title
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID
JOIN COURSE C ON E.CID = C.CID;
```

- Result: (Amal, Databases), (Amal, Networks), (Binu, Databases).
- **RA:** π Name, Title (STUDENT ⋈ ENROLL ⋈ π CID, Title (COURSE)). (COURSE is projected first so its Dept column is not matched with STUDENT's Dept by the natural join.)

Self-join (same table twice) needs two different aliases:

```
SELECT DISTINCT E1.SID
FROM ENROLL E1, ENROLL E2
WHERE E1.SID = E2.SID AND E1.CID <> E2.CID;
```

- Pairs of enrolments of the **same student** in **different courses** → students in at least two courses. Result: 1.
- **RA:** uses rename ρ to make two copies: π E1.SID (σ E1.SID=E2.SID ∧ E1.CID≠E2.CID (ρE1(ENROLL) × ρE2(ENROLL))).

0.6 Subqueries: IN and NOT IN (RA: join and difference)

A subquery is a query inside another query.

```
SELECT Name
FROM STUDENT
WHERE SID NOT IN (SELECT SID FROM ENROLL WHERE Grade = 'F');
```

- The inner query finds students who got an F: {2}. `NOT IN` keeps students whose SID is **not** in that list.
- Result: Amal, Chamal (students who never got an F).
- `IN (...)` = "is in the list" (like a join). `NOT IN (...)` = "is not in the list" → in RA this is **difference (−)**.
- **RA:** π Name ((π SID (STUDENT) − π SID (σ Grade='F' (ENROLL))) ⋈ STUDENT).
- Similar: `EXISTS (subquery)` = the subquery returns at least one row; `NOT EXISTS` = it returns none.

0.7 UNION, INTERSECT, EXCEPT (RA: ∪, ∩, −)

```
SELECT SID FROM STUDENT WHERE Dept = 'IT'
UNION
SELECT SID FROM ENROLL WHERE CID = 'C2';
```

- **UNION** = rows from either query (duplicates removed): {1, 3} ∪ {1} = {1, 3}.
- **INTERSECT** = rows in both: {1}. **EXCEPT** (MINUS in Oracle) = rows in the first but not the second: {3}.
- Both queries must return the **same number and type of columns** ("union-compatible").
- **RA:** π SID (σ Dept='IT' (STUDENT)) ∪ π SID (σ CID='C2' (ENROLL)).

0.8 Aggregates, GROUP BY, HAVING (RA: γ grouping)

```
SELECT SID, COUNT(*) AS NumCourses
FROM ENROLL
GROUP BY SID
HAVING COUNT(*) >= 2;
```

- **Aggregate functions:** `COUNT(*)` (number of rows), `SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`.
- `GROUP BY SID` = make **one group per student**, then compute COUNT for each group: (1 → 2), (2 → 1).
- `HAVING COUNT(*) >= 2` = keep only **groups** that meet the condition. Result: (1, 2).
- **WHERE vs HAVING:** WHERE filters **rows before** grouping. HAVING filters **groups after** grouping.
- `AS NumCourses` just names the output column.
- **RA:** σ NumCourses≥2 (γ SID, COUNT(*)→NumCourses (ENROLL)). γ (gamma) = group and aggregate.

0.9 ORDER BY

`ORDER BY Age DESC` sorts the result (ASC = small to big, the default; DESC = big to small). RA has **no sorting** operator, because relations are sets with no order.

0.10 The most important rule: the order SQL really runs in

You **write** SQL as SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY. The database **runs** it in this order, and this is exactly the order you build the RA expression from the **inside out**:

Step	SQL clause	What it does	RA
1	FROM (+ join conditions)	Combine the tables	× or ⋈
2	WHERE	Filter rows	σ
3	GROUP BY + aggregates	Make groups, compute COUNT/SUM…	γ
4	HAVING	Filter groups	σ (after γ)

Step	SQL clause	What it does	RA
5	SELECT	Keep the output columns	π
6	ORDER BY	Sort	(none in RA)

So `SELECT a FROM R, S WHERE R.x = S.y AND R.z > 5` becomes **π a (σ R.z>5 (R ⋈ R.x=S.y S))**: join first (inside), then filter, then project (outside).

Lesson part 1: How a database processes a query

Query processing = the steps a DBMS takes to turn an SQL string into an efficient plan, run it, and return the result. **Goal:** the **correct result** using the **least resources** (CPU time, memory, disk I/O).

1.1 The stages, in order

```
SQL query (a text string)
      |
      v
[1] Parsing (scanning + syntax check)      --> parse tree
      |
      v
[2] Semantic analysis (check the catalog)   --> validated query
      |
      v
    Translation                             --> relational algebra / query tree
      |
      v
[3] Query optimization  <-- catalog statistics (rows, blocks, indexes, distinct values)
      |
      v
[4] Execution plan generation               --> physical plan (algorithms + access paths)
      |
      v
[5] Query execution  <-- data on disk
      |
      v
[6] Result presentation                     --> rows returned to the user / application
```

1. **Parsing.**
   - **Scanning (lexical analysis):** the string is split into **tokens**: keywords (SELECT, FROM, WHERE), table and column names, operators, constants.
   - **Parsing (syntax analysis):** the tokens are checked against the **grammar** of SQL, and a **parse tree** is built. A syntax error (for example `SELEC`) **stops** the process with an error message.
2. **Semantic analysis (validation).** Checks the query **makes sense** for this database: the tables and columns **exist in the catalog**, data types match, functions and aggregates are used correctly, the user has permission. Then the query is **translated** into an internal form: a **relational algebra expression / query tree**.
3. **Query optimization.** The optimizer creates **many equivalent ways** to run the query (different orders of joins, pushing filters down, index scan vs table scan, different join algorithms), **estimates the cost** of each using **statistics from the catalog**, and **chooses the cheapest**.
4. **Execution plan generation.** The chosen strategy becomes a detailed, step-by-step **execution plan** (physical plan): which operator runs first, which algorithm each uses (for example hash

join), which index is used, and whether results are pipelined or stored.

5. **Query execution.** The **query execution engine** runs the plan: reads data from disk or memory (table scans or indexes), applies filters (WHERE), joins tables, aggregates (GROUP BY, COUNT, SUM), sorts (ORDER BY). Parts may run in parallel.
6. **Result presentation.** The result rows are formatted (sorted, grouped) and sent back to the user or application.

If the exam asks for **four** steps, write: **(1) Parsing and translation, (2) Optimization, (3) Execution plan generation, (4) Evaluation (execution) and returning results**, and mention semantic checking inside step 1.

1.2 Why relational algebra matters for query processing

- It is the **formal internal language** the query is translated into.
- RA expressions can be **rewritten into equivalent ones** using rules (for example, filter before joining), which is how the optimizer finds faster plans.
- An RA expression says **what** to compute but only partly **how**: the same expression can be run with different algorithms, so the optimizer still has choices.
- Other benefits listed in the lecture: foundation of SQL, precise (mathematical) meaning, set-based operations, simple operations that can be **combined**, **data independence** (queries do not depend on how data is stored).

1.3 Pipelining vs materialization

When a plan has several operations (for example σ then ⋈ then π), the output of one becomes the input of the next. There are two ways to pass it along:

	Materialization	Pipelining
How	Each operation runs **completely** and its whole result is **stored** (a temporary table in memory or on disk), then the next operation reads it	Each row is passed **straight to the next operation** as soon as it is produced; no temporary table
Memory and I/O	High: intermediate results are written and read again	**Low**: no intermediate results stored
First results	Only after each step finishes	**Early**: rows start coming out immediately
Works for	Any operation	Operations that can stream (σ, π, many joins); harder for sorting, set operations, some joins

Benefits of pipelining (lecture): less memory, fewer disk I/Os, faster execution, better scalability for large data.

Challenges of pipelining (lecture): some operations need a row again later (certain joins), so it must be stored anyway; set operations (∪, ∩, −) and sorting compare large parts of the data, so they are hard to pipeline; error handling and rollback are harder because there is no complete intermediate result.

Lesson part 2: Relational algebra operations

Relational algebra = a formal language of operations that take one or two **relations (tables)** as input and give a **new relation** as output. Because the output is always a relation, operations can be **nested** (the output of one is the input of the next). Examples below use the sample STUDENT, COURSE and ENROLL tables from the SQL basics section.

2.1 The basic operations

1. Select σ (filter rows) — SQL: WHERE

- Syntax: **σ condition (R)**. Keeps the rows of R where the condition is true. Same columns, fewer rows.
- Example: σ Age>21 (STUDENT) → Binu, Chamal.
- Conditions can use =, ≠, <, >, ≤, ≥ and ∧ (AND), ∨ (OR), ¬ (NOT).

2. Project π (keep columns) — SQL: SELECT DISTINCT column list

- Syntax: **π A1, A2, … (R)**. Keeps only the listed columns and **removes duplicate rows**.
- Example: π Dept (STUDENT) → {IT, CSE} (IT appears once).

3. Rename ρ — SQL: AS (alias)

- Syntax: **ρ S (R)** renames relation R to S; **ρ S(B1, B2, …) (R)** also renames its columns.
- Needed when the **same table is used twice** (self-join), or to make column names match for a natural join.

4. Cartesian product × — SQL: FROM R, S (no condition)

- Syntax: **R × S**. Every row of R paired with **every** row of S. If R has 3 rows and S has 3 rows, the result has 9 rows.
- Rarely useful alone; followed by σ it becomes a join.

5. Union ∪, Intersection ∩, Difference − — SQL: UNION, INTERSECT, EXCEPT

- **R ∪ S** = rows in R **or** S (no duplicates). **R ∩ S** = rows in **both**. **R − S** = rows in R but **not** in S.
- R and S must be **union-compatible**: the same number of columns, with matching types (so usually you **project** both sides to the same columns first).
- Example: π SID (STUDENT) − π SID (ENROLL) = {1, 2, 3} − {1, 2} = {3} → students with no enrolment.
- ∪ and ∩ are commutative (order does not matter). **− is not**: R − S ≠ S − R.

2.2 Joins

Join	Symbol	Meaning	SQL
Theta join	R ⋈θ S	Pairs of rows that satisfy any condition θ (e.g. R.A < S.B). Same as σθ(R × S)	`JOIN … ON condition`

Join	Symbol	Meaning	SQL
Equi-join	R ⋈ R.A=S.B S	A theta join where the condition uses only **=**	`JOIN … ON R.A = S.B`
Natural join	R ⋈ S	Equi-join on **all columns with the same name**, and the duplicate column is shown once	`NATURAL JOIN`
Left / right / full outer join	⟕ ⟖ ⟗	Like a join, but also keeps rows with **no match** from the left / right / both tables, filling the missing columns with NULL	`LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN`

- Example: STUDENT ⋈ ENROLL (natural join on SID) → (1, Amal, IT, 21, C1, A), (1, Amal, IT, 21, C2, B), (2, Binu, CSE, 23, C1, F). Chamal disappears (no match). With a **left outer join**, Chamal would stay with CID = NULL, Grade = NULL.
- **Natural join trap:** it matches **every** column with the same name. STUDENT and COURSE both have **Dept**, so STUDENT ⋈ ENROLL ⋈ COURSE would also force student Dept = course Dept. Fix: **project first** (π CID, Title (COURSE)) or use an explicit condition.
- **Join vs theta join (lecture exercise):** a theta join allows **any** comparison (<, >, ≠, =); an equi-join only =; a natural join is an equi-join on same-named columns with duplicates removed.

2.3 Division ÷ (for "all" / "every" questions)

- **R(A, B) ÷ S(B)** = the A-values in R that are paired with **every** B-value in S.
- Example: A = π SID, CID (ENROLL) = {(1, C1), (1, C2), (2, C1)}.
  - A ÷ {C1} = {1, 2} (both took C1).
  - A ÷ {C1, C2} = {1} (only student 1 took **both**).
- Use it for wording like "enrolled in **all** IT courses", "supplies **every** red part".
- Without ÷ (using basic operations): **π A(R) − π A((π A(R) × S) − R)**. In words: all candidates, minus those who are **missing** at least one required value.

2.4 Aggregation γ (grouping)

- **γ grouping-columns, function(column)→name (R)**, for example γ SID, COUNT(*)→Total (ENROLL).
- Groups the rows, then computes COUNT, SUM, AVG, MIN, MAX for each group. Same as SQL **GROUP BY** with aggregates.
- After γ, only the **grouping columns and the new aggregate columns exist**. So a filter on any other column must be applied **before** γ (like WHERE), and a filter on the aggregate **after** γ (like HAVING).

2.5 NULL values and multiset (bag) relational algebra

- **NULL** = unknown or missing value. Any comparison with NULL (NULL = 5, NULL > 3) is **unknown**, so σ does **not** keep that row. Rows with NULL in the join column never match in a join. Aggregates (SUM, AVG, MAX…) **ignore** NULLs; COUNT(*) counts all rows. Outer joins **create** NULLs for unmatched rows.

- **Multiset (bag) RA:** normal RA treats relations as **sets** (no duplicates). Multiset RA allows **duplicate rows**, like real SQL (which keeps duplicates unless you write DISTINCT). **Use:** it models SQL exactly, gives correct counts and sums (duplicates matter for COUNT and SUM), and is **cheaper** to run because removing duplicates needs sorting or hashing.

2.6 Comparing operations (lecture exercise)

Pair	Difference
Selection σ vs Projection π	σ picks **rows** (horizontal subset, same columns); π picks **columns** (vertical subset, duplicates removed)
Union ∪ vs Intersection ∩	∪ = rows in **either** relation; ∩ = rows in **both**. Both need union-compatible inputs
Join vs Theta join	Natural/equi-join matches with **=** only; theta join allows **any** comparison

2.7 Operator precedence and associativity

The lecture does not give a precedence table; this is the usual textbook convention. **Always use brackets** in your answers so there is no doubt.

- **Precedence (highest first):** (1) unary operators σ, π, ρ (they apply to the expression right after them); (2) × and joins; (3) ∩; (4) ∪ and − (equal, worked **left to right**).
- Example: π name σ age>20 S ⋈ R ∪ T is read as ((π name (σ age>20 (S))) ⋈ R) ∪ T.
- **Associative** (brackets can move): ×, ⋈, ∪, ∩. (R ⋈ S) ⋈ T = R ⋈ (S ⋈ T). Together with commutativity this lets the optimizer **reorder joins**.
- **Not associative:** −. Example: R = {1, 2}, S = {1}, T = {2}. (R − S) − T = {2} − {2} = ∅, but R − (S − T) = {1, 2} − {1} = {2}.

2.8 RA and database design (lecture exercise)

- **Normalization** splits a table into smaller tables using **projection** (decomposition), e.g. EMP_DEPT(EmpID, Name, DeptID, DeptName) → π EmpID, Name, DeptID (EMP_DEPT) and π DeptID, DeptName (EMP_DEPT).
- A decomposition is **lossless** if the **natural join** of the parts gives back exactly the original table.
- **Constraints** can be written in RA: a foreign key R[B] ⊆ S[B] is π B(R) − π B(S) = ∅ (see Tutorial 3 Q3).

Cheat box: relational algebra at a glance

Symbol table

Symbol	Name	What it does	SQL	Example
σ	Select	Keep rows that match a condition	WHERE	σ Dept='IT' (STUDENT)
π	Project	Keep columns, remove duplicates	SELECT DISTINCT	π Name (STUDENT)
ρ	Rename	New name for a table or its columns	AS (alias)	ρ E1 (ENROLL)
×	Cartesian product	Every row with every row	FROM R, S	STUDENT × COURSE
⋈	Natural join	Match on same-named columns	NATURAL JOIN	STUDENT ⋈ ENROLL
⋈θ	Theta / equi-join	Match on a given condition	JOIN ... ON	R ⋈ R.a=S.d S
⟕ ⟖ ⟗	Left / right / full outer join	Join + keep unmatched rows (NULLs)	LEFT / RIGHT / FULL JOIN	STUDENT ⟕ ENROLL
∪	Union	In either	UNION	R ∪ S
∩	Intersection	In both	INTERSECT	R ∩ S
−	Difference	In first, not in second	EXCEPT, NOT IN, NOT EXISTS	R − S
÷	Division	Paired with **all** values of S	(double NOT EXISTS)	R ÷ S
γ	Group / aggregate	Groups + COUNT, SUM, AVG, MIN, MAX	GROUP BY	γ SID, COUNT(*)→n (ENROLL)
∧ ∨ ¬	And, or, not	Combine conditions	AND, OR, NOT	σ a>1 ∧ b<5 (R)

SQL clause → RA (build from the inside out)

SQL	RA	Position
FROM R, S + join condition	R ⋈ condition S	Innermost
WHERE row condition	σ condition	Around the join
GROUP BY + aggregates	γ	Around σ
HAVING	σ on the aggregate	Around γ
SELECT columns	π columns	Outermost

SQL	RA	Position
NOT IN / NOT EXISTS (subquery)	outer − subquery	Combine two projections
IN / EXISTS (subquery)	⋈ (or ∩)	Combine
UNION / INTERSECT / EXCEPT	∪ / ∩ / −	Combine

Template: `SELECT cols FROM R, S WHERE join AND filter` → **π cols (σ filter (R ⋈ join S))**.

English wording → operator

If the question says...	Use	Pattern
"find / list X where ..."	σ then π	π X (σ condition (R))
"... who works for / enrolled in / reserved ..." (information in 2 tables)	⋈	π X (σ condition (R) ⋈ S)
"at least one"	⋈	join with the linking table
"or" (same table)	∨ inside σ	σ c1 ∨ c2 (R)
"or" (different tables)	∪	π id (...) ∪ π id (...)
"and" across two different rows (a red part **and** a green part)	∩ (or ⋈ on the id)	π id (red...) ∩ π id (green...)
"not", "never", "does not", "no"	−	π id (all) − π id (those who do)
"all", "every"	÷	π id, item (R) ÷ π item (required)
"at least two different"	self-join with ρ and ≠	σ R1.id=R2.id ∧ R1.x≠R2.x (ρR1(R) × ρR2(R))
"only"	−	(those who do) − (those who do something else)
"same city as ...", "same street as their manager"	natural join on the shared column, or ρ + σ equal	see Tutorial 3 (d), (f)
"how many", "total", "average"	γ	γ group, COUNT(*)→n (R)

Key facts to remember

- Read RA from the **inside out**. Write it from the inside out too: tables → join → filter → project.
- π removes duplicates; relations are **sets**.
- ∪, ∩, − need **union-compatible** inputs (same columns): project both sides first.
- **Subtract IDs, not names** (two people can share a name), then join back to get the names.
- Natural join matches **all** same-named columns: project away extra ones (like Dept) first.
- π distributes over ∪ but **not** over −.
- After γ only the grouping columns and aggregates exist.

How-to-answer recipes

Recipe 1: English → RA with Filter → Join → Reduce ★

This is the memory rule given in Tutorial 5.

1. **Output:** which columns does the question want? (These go in the outer π.)
2. **Tables:** which tables hold the columns and the conditions?
3. **Filter (σ):** apply each condition to its own table **first** (small tables are faster to join).
4. **Join (⋈):** connect the tables on their linking columns (foreign keys). Project away same-named columns you do not want matched.
5. **Reduce (π):** keep only the output columns.
6. **Check:** read it back in English. Does every condition appear? Could a natural join match an extra column?

Worked example: *names of students who got grade A in a CSE course* (Tutorial 5 vii).

- Output: Name (STUDENT). Tables: STUDENT, ENROLL, COURSE.
- Filter: σ Grade='A' (ENROLL), σ Dept='CSE' (COURSE).
- Join: on CID, then on SID. Reduce: π Name.
- **π Name (STUDENT ⋈ π SID (σ Grade='A' (ENROLL) ⋈ π CID (σ Dept='CSE' (COURSE))))**

SQL for the same query:

```
SELECT DISTINCT S.Name
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID
JOIN COURSE C ON E.CID = C.CID
WHERE E.Grade = 'A' AND C.Dept = 'CSE';
```

Recipe 2: "not", "never", "does not" → difference ★

1. Find **everyone** (the IDs): π ID (main table).
2. Find those who **do** the thing (the IDs): π ID (...condition...).
3. **Subtract:** everyone − those who do.
4. If names are wanted, **join back** to the main table and project the name.

Why subtract IDs and not names? Two people can have the same name; subtracting names could remove the wrong person.

Trap: "not working for City Bank" is **not** σ cname≠'City Bank'. That selection still returns a person who works for City Bank **and** another company.

Recipe 3: "all" / "every" → division

1. Left side: π ID, item (the linking table), e.g. π SID, CID (ENROLL).
2. Right side: π item (the required set), e.g. π CID (σ Dept='IT' (COURSE)).
3. Answer = left ÷ right, then join back for names.

4. If ÷ is not allowed: Missing = π ID ((π ID (left) × right) − left); answer = π ID (left) − Missing.

Recipe 4: "at least two different" → self-join

1. Make two copies with rename: ρ R1 (R), ρ R2 (R).
2. Product: ρ R1 (R) × ρ R2 (R).
3. Keep pairs with the **same person** and a **different item**: σ R1.ID = R2.ID ∧ R1.item ≠ R2.item.
4. Project R1.ID (and join back for names).

Recipe 5: SQL → RA (and checking a given translation)

1. Translate the FROM tables and join conditions into ⋈ (innermost).
2. Wrap the other WHERE conditions in σ.
3. If there is GROUP BY, wrap in γ (grouping columns + aggregates).
4. HAVING becomes σ **after** γ.
5. SELECT becomes the outer π.
6. A subquery with NOT IN → translate it separately, then use −. With IN → join.
7. To **check** a given RA translation: is every clause there, in this order? A WHERE condition placed **after** γ is **wrong** (those columns no longer exist after grouping).

Recipe 6: "Describe what this RA returns" and "compare two expressions"

1. Read **inside out** and translate each operator into English (σ = "rows where...", ⋈ = "matched with...", − = "except those who...").
2. Write one plain sentence for the whole result.
3. **Similarities:** same tables / same conditions / same output column.
4. **Differences:** which operators differ (− vs σ, × with ≠, ∪), what each returns, and give a person who would be in one result but not the other.

Recipe 7: "Steps in evaluating an SQL query" ★

Draw the flow (SQL string → parse → translate → optimize → plan → execute → result). For each step write **what happens** and **what it produces**, with one example (a syntax error caught by the parser, a missing column caught by validation, an index chosen by the optimizer).

Tutorial 3: Relational model and relational algebra (SQL + RA)

> **Exam link:** Q4 (a)–(f) came in the **2020/21 paper as Q7(d), word for word**, and almost the same in **2022/23 Q4(c)** (with "First Bank Corporation" instead of City Bank).

Q1. What is a relation, and what two properties must its elements have?

A **relation** is a **table**: a **set of tuples (rows)** over a set of attributes (columns), where each value comes from the attribute's **domain** (allowed values).

The two properties of its elements (tuples):

1. **Tuples are distinct:** a relation is a **set**, so there are **no duplicate rows** (this is why every relation has a key).
2. **Values are atomic:** each cell holds **one single, indivisible value** (first normal form).

(Also: the order of rows and the order of columns do not matter.)

Q2. Relation vs relation schema

Relation schema	Relation (instance)
The **definition / structure**: relation name, attributes with domains, and keys, e.g. N1({A, B}, {A})	The **set of tuples** stored for that schema at one moment
Changes rarely (at design time)	Changes with every insert, delete, update
Like a table's column headings	Like the table's current rows

Q3. r1 over N1({A, B}, {A}) and r2 over N2({B, C}, {B}). Express N1[B] ⊆ N2[B] using projection.

The constraint means: every B value in r1 must also appear as a B value in r2 (B in N1 is a **foreign key** pointing to N2, whose key is B).

> **RA:** π B (r1) ⊆ π B (r2), or equivalently **π B (r1) − π B (r2) = ø** (nothing is left over).

Q4. RA queries

Schema (short names: pname, cname, mname):

```
Lives(pname, street, city)          Works(pname, cname, salary)
Located_in(cname, city)             Manages(pname, mname)
```

Watch out: **city** is in both Lives and Located_in, so a natural join of those two also matches city.

(a) Names of all employees who work for City Bank.

- Filter Works to City Bank rows, keep the names.

```
SELECT pname
FROM Works
WHERE cname = 'City Bank';
```

> **RA:** π pname (σ cname='City Bank' (Works))

(b) Name and city of all employees who work for City Bank.

- The city is in Lives, so join on the person name.

```
SELECT W.pname, L.city
FROM Works W
JOIN Lives L ON W.pname = L.pname
WHERE W.cname = 'City Bank';
```

> **RA:** π pname, city (σ cname='City Bank' (Works) ⋈ Lives)

(c) Name, street and city of all employees who work for City Bank and earn more than $10,000.

```
SELECT L.pname, L.street, L.city
FROM Works W
JOIN Lives L ON W.pname = L.pname
WHERE W.cname = 'City Bank' AND W.salary > 10000;
```

> **RA:** π pname, street, city (σ cname='City Bank' ∧ salary>10000 (Works) ⋈ Lives)

(d) All employees who live in the same city as the company they work for.

- Lives ⋈ Works matches on pname. Then joining Located_in matches on **cname and city** (both names are shared), so only rows where the person's city = the company's city survive.

```
SELECT DISTINCT L.pname
FROM Lives L
JOIN Works W      ON L.pname = W.pname
JOIN Located_in C ON W.cname = C.cname
WHERE L.city = C.city;
```

> **RA:** π pname (Lives ⋈ Works ⋈ Located_in)

With explicit conditions: π Lives.pname (σ Lives.city = Located_in.city (Lives ⋈ Lives.pname=Works.pname Works ⋈ Works.cname=Located_in.cname Located_in))

(e) All persons who do not work for City Bank. (Hint: set difference.)

- All persons (Lives lists everyone) **minus** those who work for City Bank.

```
SELECT pname
FROM Lives
WHERE pname NOT IN (SELECT pname FROM Works WHERE cname = 'City Bank');
```

> **RA:** π pname (Lives) − π pname (σ cname='City Bank' (Works))

- Do **not** write π pname (σ cname≠'City Bank' (Works)): it would include someone who works for City Bank **and** another company.

(f) All employees who live in the same city and on the same street as their manager. (Hint: use Lives twice, with rename.)

- One copy of Lives for the employee, a renamed copy for the manager. Match employee → Manages → manager, then require the same street and city.

```
SELECT E.pname
FROM Lives E
JOIN Manages M ON E.pname = M.pname
JOIN Lives MG  ON M.mname = MG.pname
WHERE E.street = MG.street AND E.city = MG.city;
```

> **RA (short form):** π pname (Lives ⋈ Manages ⋈ ρ M(mname, street, city) (Lives))

- ρ renames the second copy of Lives so its person column is called **mname**. The natural join then matches the manager's name **and** street **and** city.

> **RA (explicit form):** π E.pname (σ E.street=MG.street ∧ E.city=MG.city (ρ E (Lives) ⋈ E.pname=Manages.pname Manages ⋈ Manages.mname=MG.pname ρ MG (Lives)))

Tutorial 5: Relational algebra practice (SQL + RA)

> **Memory rule from the tutorial:** Filter → Join → Reduce (project). **Exam link:** the "not" (difference) and "at least two different" (self-join) patterns came in **2023/24 Q4(c)**.

```
STUDENT(SID, Name, Dept, Age)     COURSE(CID, Title, Dept)     ENROLL(SID, CID, Grade)
FACULTY(FID, Name, Dept)          TEACHES(FID, CID)
```

Natural-join traps here: Dept is in STUDENT, COURSE and FACULTY; Name is in STUDENT and FACULTY. Project away the extra columns **before** joining (done below), or the natural join would also force them to be equal.

(i) List all student names.

```
SELECT DISTINCT Name FROM STUDENT;
```

> **RA:** π Name (STUDENT)

(ii) Student IDs of students in the "IT" department.

```
SELECT SID FROM STUDENT WHERE Dept = 'IT';
```

> **RA:** π SID (σ Dept='IT' (STUDENT))

(iii) Course IDs of courses taught by faculty from the "CSE" department.

Filter FACULTY to CSE → join TEACHES on FID → keep CID.

```
SELECT DISTINCT T.CID
FROM FACULTY F
JOIN TEACHES T ON F.FID = T.FID
WHERE F.Dept = 'CSE';
```

> **RA:** π CID (π FID (σ Dept='CSE' (FACULTY)) ⋈ TEACHES)

(iv) Names of students enrolled in at least one course.

"At least one" = just join with ENROLL.

```
SELECT DISTINCT S.Name
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID;
```

> **RA:** π Name (STUDENT ⋈ π SID (ENROLL))

(v) Course titles taken by students from the "ECE" department.

```
SELECT DISTINCT C.Title
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID
JOIN COURSE C ON E.CID = C.CID
WHERE S.Dept = 'ECE';
```

> **RA:** π Title (π SID (σ Dept='ECE' (STUDENT)) ⋈ ENROLL ⋈ π CID, Title (COURSE))

(COURSE is projected first so its Dept is not matched with the student's Dept.)

(vi) Names of faculty who do not teach any course.

"Not" → difference on the IDs, then join back for the names.

```
SELECT Name
FROM FACULTY
WHERE FID NOT IN (SELECT FID FROM TEACHES);
```

> **RA:** π Name ((π FID (FACULTY) − π FID (TEACHES)) ⋈ FACULTY)

(vii) Names of students who received grade 'A' in courses offered by the "CSE" department.

```
SELECT DISTINCT S.Name
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID
JOIN COURSE C ON E.CID = C.CID
WHERE E.Grade = 'A' AND C.Dept = 'CSE';
```

> **RA:** π Name (STUDENT ⋈ π SID (σ Grade='A' (ENROLL) ⋈ π CID (σ Dept='CSE' (COURSE))))

(viii) Names of students enrolled in all courses offered by the "IT" department.

"All" → division. In SQL the easiest way is to **count**: the student's number of IT courses must equal the total number of IT courses.

```
SELECT S.Name
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID
JOIN COURSE C ON E.CID = C.CID
WHERE C.Dept = 'IT'
GROUP BY S.SID, S.Name
HAVING COUNT(DISTINCT C.CID) = (SELECT COUNT(*) FROM COURSE WHERE Dept = 'IT');
```

> **RA:** π Name (STUDENT ⋈ (π SID, CID (ENROLL) ÷ π CID (σ Dept='IT' (COURSE))))

Without ÷: let A = π SID (ENROLL) and C = π CID (σ Dept='IT' (COURSE)). Missing = π SID ((A × C) − π SID, CID (ENROLL)). Answer = π Name ((A − Missing) ⋈ STUDENT).

(ix) Names of students enrolled in at least two different courses.

Self-join: two enrolments of the **same** student in **different** courses.

```
SELECT DISTINCT S.Name
FROM STUDENT S
JOIN ENROLL E1 ON S.SID = E1.SID
JOIN ENROLL E2 ON E1.SID = E2.SID
WHERE E1.CID <> E2.CID;
```

(Another way: `GROUP BY S.SID, S.Name HAVING COUNT(DISTINCT E.CID) >= 2`.)

> **RA:** Two = ρ Two(SID) (π E1.SID (σ E1.SID=E2.SID ∧ E1.CID≠E2.CID (ρ E1 (ENROLL) × ρ E2 (ENROLL)))). Answer = π Name (STUDENT ⋈ Two).

(x) Student names who have never received grade 'F'.

"Never" → difference.

```
SELECT Name
FROM STUDENT
WHERE SID NOT IN (SELECT SID FROM ENROLL WHERE Grade = 'F');
```

> **RA:** π Name ((π SID (STUDENT) − π SID (σ Grade='F' (ENROLL))) ⋈ STUDENT)

(This includes students with no enrolments. If only enrolled students are meant, start from π SID (ENROLL) instead.)

(xi) Course titles not taken by any student.

```
SELECT Title
FROM COURSE
WHERE CID NOT IN (SELECT CID FROM ENROLL);
```

> **RA:** π Title ((π CID (COURSE) − π CID (ENROLL)) ⋈ COURSE)

(xii) Names of faculty who teach only CSE department courses.

"Only" = faculty who teach something, **minus** those who teach a non-CSE course.

```
SELECT Name
FROM FACULTY
WHERE FID IN (SELECT FID FROM TEACHES)
  AND FID NOT IN (SELECT T.FID
                  FROM TEACHES T
                  JOIN COURSE C ON T.CID = C.CID
                  WHERE C.Dept <> 'CSE');
```

> **RA:** T = π FID (TEACHES). NonCSE = π FID (TEACHES ⋈ π CID (σ Dept≠'CSE' (COURSE))). Answer = π Name ((T − NonCSE) ⋈ FACULTY)

(xiii) Student names enrolled in every course taught by faculty from the ECE department.

"Every" → division. First find the set of courses taught by ECE faculty.

```
SELECT S.Name
FROM STUDENT S
JOIN ENROLL E ON S.SID = E.SID
WHERE E.CID IN (SELECT T.CID FROM TEACHES T
                JOIN FACULTY F ON T.FID = F.FID
                WHERE F.Dept = 'ECE')
GROUP BY S.SID, S.Name
HAVING COUNT(DISTINCT E.CID) = (SELECT COUNT(DISTINCT T.CID) FROM TEACHES T
                                JOIN FACULTY F ON T.FID = F.FID
                                WHERE F.Dept = 'ECE');
```

> **RA:** C_ECE = π CID (π FID (σ Dept='ECE' (FACULTY)) ⋈ TEACHES). Answer = π Name (STUDENT ⋈ (π SID, CID (ENROLL) ÷ C_ECE))

Pattern summary

Wording	Operator	Queries above
"at least one"	join	(iv)
"not", "never", "no"	difference	(vi), (x), (xi)
"all", "every"	division	(viii), (xiii)
"at least two different"	self-join with ρ and ≠	(ix)
"only"	(do) − (do something else)	(xii)

Lesson exercise (from the slides)

Exercise 1. Employee(EmpID, Name, Age, DeptID), Department(DeptID, DeptName), Project(ProjectID, DeptID, ProjectName), Assignment(EmpID, ProjectID). Find the names of employees who work on a project in the "IT" department and are older than 30.

- Filter employees by age; find IT projects (Project joined with the IT department); link through Assignment.
- **Trap:** Employee and Project both have **DeptID**. The project's department matters here, not the employee's, so project the IT projects down to **ProjectID** before joining with the employee side.

```
SELECT DISTINCT E.Name
FROM Employee E
JOIN Assignment A ON E.EmpID = A.EmpID
JOIN Project P    ON A.ProjectID = P.ProjectID
JOIN Department D ON P.DeptID = D.DeptID
WHERE D.DeptName = 'IT' AND E.Age > 30;
```

> **RA:** π Name (σ Age>30 (Employee) ⋈ Assignment ⋈ π ProjectID (Project ⋈ σ DeptName='IT' (Department)))

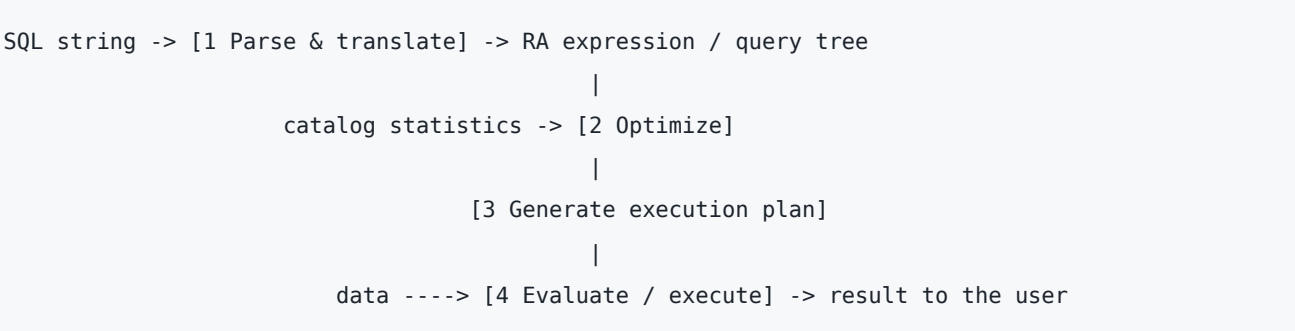
Exercises 2–5 on the slide (NULLs, aggregates, division, multiset RA, union vs intersection, join vs theta join, selection vs projection, RA in normalization) are answered in **Lesson part 2**, sections 2.3 to 2.8.

Past paper questions with answers

Grouped by topic. ★ = asked in more than one paper.

A. Query processing steps

> **2020/21 Q1(c) (30m), 2022/23 Q4(a) (25m) and 2023/24 Q4(a) (25m) ★** A user connected to a database server and submitted an SQL query as a string. Give the (four) important steps involved in evaluating the query, and the order in which they are performed.



1. **Parsing and translation:**
   - **Scanning:** the string is split into **tokens** (SELECT, FROM, WHERE, table and column names, operators, constants).
   - **Parsing:** tokens are checked against SQL **grammar**; a **parse tree** is built. A syntax error stops processing.
   - **Validation (semantic analysis):** table and column names are checked against the **catalog**, data types and privileges are checked.
   - **Translation** into an internal form: a **relational algebra expression / query tree**.
2. **Query optimization:** the optimizer builds **equivalent RA expressions** (push selections down, turn × + σ into ⋈, reorder joins) and alternative **access paths and algorithms** (index scan vs table scan, hash join vs nested-loop join), estimates each one's cost from **catalog statistics** (number of rows, blocks, distinct values, indexes), and picks the **cheapest plan**.
3. **Execution plan generation:** the chosen plan becomes a **physical plan**: an ordered tree of operators with the algorithm and access method for each (and whether results are pipelined or materialized).
4. **Evaluation (execution):** the **query execution engine** runs the plan (scans, filters, joins, aggregates, sorts) and **returns the result** to the user or application.

Order: parse → validate / translate → optimize → generate plan → execute → return results. Optimization always comes before execution, and it is the only step that decides **how** the query runs.

2023/24 Q5(a) (20m). Explain the process of query processing in a DBMS: its stages, techniques, and how they make execution efficient.

Stages (lecture names): (1) parsing, (2) semantic analysis, (3) query optimization, (4) execution plan generation, (5) query execution, (6) result presentation.

Stage	Technique	How it helps efficiency
Parsing / translation	Parse tree → relational algebra / query tree	A formal structure that can be safely rewritten using equivalence rules
Optimization (logical)	**Heuristics:** push selections and projections down, replace × + σ with ⋈, do the most restrictive joins first	Makes intermediate results small early; avoids Cartesian products
Optimization (cost-based)	**Cost models** with selectivity and size estimates; choosing the join order	Picks the cheapest access paths and join order
Plan generation	Access paths (**index scan** vs table scan); join algorithms (nested loop, index nested loop, hash, sort-merge)	Fewer disk I/Os and less CPU
Execution	**Pipelining** (rows flow between operators without temporary tables), parallel execution, buffering	Less memory and I/O, faster first results

Conclusion: the correct result with **minimum resource use** (CPU, memory, I/O), which is the goal of query processing.

B. Translating SQL into relational algebra

2020/21 Q1(a) (40m). Tables R(a, b, c), S(d, e, f), T(g, h, i). For the SQL query below, say whether each RA expression is a correct translation (γ = grouping/aggregation).

```
SELECT R.b
FROM R, S, T
WHERE R.a = S.d AND S.e = T.g AND T.h > 21 AND S.f < 50
GROUP BY R.b
HAVING count(*) > 2;
```

- (i) π R.b (σ TOTAL>2 (γ R.b, count(*)→TOTAL (σ T.h>21 ∧ S.f<50 (R ⋈ R.a=S.d (S ⋈ S.e=T.g T)))))
- (ii) π R.b (σ TOTAL>2 (σ T.h>21 ∧ S.f<50 (γ R.b, count(*)→TOTAL (R ⋈ R.a=S.d (S ⋈ S.e=T.g T)))))

SQL's real order is: FROM (joins) → WHERE (filter rows) → GROUP BY (γ) → HAVING (filter groups) → SELECT (π).

(i) is correct. The joins S ⋈ S.e=T.g T and R ⋈ R.a=S.d (...) are the FROM clause with its join conditions. σ T.h>21 ∧ S.f<50 applies the other WHERE conditions to rows **before grouping**. γ groups by R.b and counts each group (GROUP BY). σ TOTAL>2 is the HAVING clause. π R.b is the SELECT. Every clause is there, in the right order.

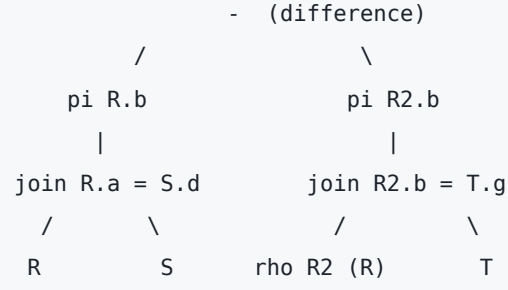
(ii) is incorrect. The filter on T.h > 21 and S.f < 50 is placed **after** γ. After grouping, the result only has the columns **R.b and TOTAL**, so T.h and S.f **no longer exist**: the expression is not even valid. And in meaning, the counts would include rows that should have been removed **before** grouping, so TOTAL and the HAVING test would be wrong. A selection on non-grouping columns **cannot be moved past γ**.

> **2020/21 Q1(b) (30m).** Give an equivalent RA expression (or expression tree / logical plan) for:

```
SELECT R.b
FROM R, S
WHERE R.a = S.d
  AND R.b NOT IN (SELECT R2.b FROM R AS R2, T WHERE R2.b = T.g);
```

1. Outer part: the b values of R rows that join with S: π R.b (R ⋈ R.a=S.d S).
2. Subquery: the b values of R (renamed R2) that match some T.g: π R2.b (ρ R2 (R) ⋈ R2.b=T.g T).
3. **NOT IN → set difference.**

> **RA:** π R.b (R ⋈ R.a=S.d S) − π R2.b (ρ R2 (R) ⋈ R2.b=T.g T)



Both sides have the single column b, so they are union-compatible and the difference is valid.

C. Relational algebra theory

> **2020/21 Q7(b) (25m).** Discuss the operator precedence and associativity of RA operators, with examples.

(The lecture gives no precedence table; this is the usual convention. Always use brackets.)

- **Precedence (highest first):** (1) unary operators σ, π, ρ — they apply to the expression right after them, so π A σ c R = π A (σ c (R)); (2) × and joins; (3) ∩; (4) ∪ and − (equal, evaluated **left to right**).

- Example: π name σ age>20 S ⋈ R ∪ T is read as ((π name (σ age>20 (S))) ⋈ R) ∪ T.
- **Associative:** ×, ⋈, ∪, ∩. (R ⋈ S) ⋈ T = R ⋈ (S ⋈ T); (R ∪ S) ∪ T = R ∪ (S ∪ T). With commutativity this lets the optimizer **reorder joins**.
- **Not associative (and not commutative):** −. Example: R = {1, 2}, S = {1}, T = {2}: (R − S) − T = ∅, but R − (S − T) = {2}.
- Unary operators nest from right to left; a cascade of σ can be reordered or merged; a cascade of π reduces to the outermost π.
- **Why it matters:** the reading decides the meaning, and associativity/commutativity are what make join reordering in optimization valid.

> **2020/21 Q7(c) (25m).** Is π A (E1 − E2) equal to π A (E1) − π A (E2)? Discuss.

No, not in general. Projection distributes over **union** but **not over difference**.

Counter-example with columns (A, B): E1 = {(a1, b1)}, E2 = {(a1, b2)}.

- Left side: E1 − E2 = {(a1, b1)} (the tuples are different) → π A = **{a1}**.
- Right side: π A (E1) = {a1}, π A (E2) = {a1} → {a1} − {a1} = **∅**.
- {a1} ≠ ∅.

Reason: projection can make **different tuples look the same** (it throws away B). Subtracting after projecting removes an A value whenever **any** tuple in E2 has it.

In general: π A (E1) − π A (E2) ⊆ π A (E1 − E2). They are equal when A contains a **key** (projection does not merge different tuples). So an optimizer may push π through ∪ but **must not** push π through −.

> **2021/22 Q5(a) (20m).** Suppliers(sID, sName, address), Parts(pID, pName, colour), Catalog(sID, pID, price); Catalog[sID] ⊆ Suppliers[sID], Catalog[pID] ⊆ Parts[pID]. If sID is a key for Suppliers, could it be a key for Catalog? Discuss.

No (in general). In Suppliers, sID is unique: one row per supplier. In Catalog, sID is a **foreign key** pointing to Suppliers. A supplier normally supplies **many parts**, so the same sID appears in many Catalog rows, e.g. (s1, p1, 10) and (s1, p2, 25): sID is **not unique** there. The key of Catalog is the **composite key (sID, pID)** (a supplier lists each part once, with one price). sID could only be a key of Catalog if each supplier supplied **at most one** part. Being a key is a property **of a relation**, not of a column name: unique in Suppliers says nothing about Catalog.

D. Writing relational algebra queries ★

> **2020/21 Q7(d)(i)–(vi) (5m × 6).** lives(person-name, street, city), works(person-name, company-name, salary), located-in(company-name, city), manages(person-name, manager-name). This is **exactly Tutorial 3 Q4**; full explanations and SQL are in the Tutorial 3 section. Answers:

- (i) Work for City Bank: π pname (σ cname='City Bank' (works))

- (ii) Name and city: π pname, city (σ cname='City Bank' (works) ⋈ lives)
- (iii) Name, street, city, earn > $10,000: π pname, street, city (σ cname='City Bank' ∧ salary>10000 (works) ⋈ lives)
- (iv) Same city as their company: π pname (lives ⋈ works ⋈ located-in)
- (v) Do not work for City Bank: π pname (lives) − π pname (σ cname='City Bank' (works))
- (vi) Same street and city as their manager: π pname (lives ⋈ manages ⋈ ρ M(mname, street, city) (lives))

2021/22 Q5(b) (80m). Suppliers(sID, sName, address), Parts(pID, pName, colour), Catalog(sID, pID, price). Use only select, project, Cartesian product and natural join.

(i) Names of all red parts.

```
SELECT pName FROM Parts WHERE colour = 'red';
```

RA: π pName (σ colour='red' (Parts))

(ii) All prices for parts that are red or green.

```
SELECT C.price
FROM Parts P JOIN Catalog C ON P.pID = C.pID
WHERE P.colour = 'red' OR P.colour = 'green';
```

RA: π price (σ colour='red' ∨ colour='green' (Parts) ⋈ Catalog)

(iii) sIDs of suppliers who supply a red or green part.

```
SELECT DISTINCT C.sID
FROM Parts P JOIN Catalog C ON P.pID = C.pID
WHERE P.colour = 'red' OR P.colour = 'green';
```

RA: π sID (σ colour='red' ∨ colour='green' (Parts) ⋈ Catalog)

(iv) sIDs of suppliers who supply a red and a green part.

- One part cannot be both colours, so this means a supplier with **a red part and also a green part** (two different rows). ∩ is not allowed here, so use a **natural join on sID**: joining two one-column relations keeps only values in both.

```
SELECT DISTINCT C.sID
FROM Catalog C JOIN Parts P ON C.pID = P.pID
WHERE P.colour = 'red'
  AND C.sID IN (SELECT C2.sID
                FROM Catalog C2 JOIN Parts P2 ON C2.pID = P2.pID
                WHERE P2.colour = 'green');
```

RA: π sID (σ colour='red' (Parts) ⋈ Catalog) ⋈ π sID (σ colour='green' (Parts) ⋈ Catalog)

(v) Names of suppliers who supply a red or green part.

```
SELECT DISTINCT S.sName
FROM Parts P
JOIN Catalog C   ON P.pID = C.pID
JOIN Suppliers S ON C.sID = S.sID
WHERE P.colour = 'red' OR P.colour = 'green';
```

RA: π sName (σ colour='red' ∨ colour='green' (Parts) ⋈ Catalog ⋈ Suppliers)

2022/23 Q4(c)(i) (3 × 10m). employee(person-name, street, city), works(person-name, company-name, salary), company(company-name, city), manages(person-name, manager-name). (Same patterns as Tutorial 3, with First Bank Corporation.)

1. Names of all employees who work for First Bank Corporation.

```
SELECT person_name FROM works WHERE company_name = 'First Bank Corporation';
```

RA: π person-name (σ company-name='First Bank Corporation' (works))

2. Names and cities of residence of all employees who work for First Bank Corporation.

```
SELECT E.person_name, E.city
FROM employee E JOIN works W ON E.person_name = W.person_name
WHERE W.company_name = 'First Bank Corporation';
```

RA: π person-name, city (employee ⋈ σ company-name='First Bank Corporation' (works))

3. Names, street addresses and cities of employees who work for First Bank Corporation and earn more than $10,000 per annum.

```
SELECT E.person_name, E.street, E.city
FROM employee E JOIN works W ON E.person_name = W.person_name
WHERE W.company_name = 'First Bank Corporation' AND W.salary > 10000;
```

> **RA:** π person-name, street, city (σ company-name='First Bank Corporation' ∧ salary>10000 (works) ⋈ employee)

> **2022/23 Q4(c)(ii) (20m).** Describe what each expression returns, and discuss similarities and differences.

- 1. π person-name (σ company-name ≠ 'First Bank Corporation' (works))
- 2. π person-name (employee) − π person-name (σ company-name ≠ 'First Bank Corporation' (works))

Expression 1: names of people who work for **at least one company other than** First Bank Corporation (FBC).

Expression 2: all employees **minus** those who work for some company other than FBC = employees who **do not work for any company other than FBC**: those who work **only for FBC**, plus employees who work for **no company at all**.

Similarities: both use the same selection σ company-name ≠ FBC (works); both return a set of person names.

Differences: (1) is a simple select + project; (2) uses **set difference**. (2) is the **complement** of (1) within the employee relation, so the two results never overlap and together cover every employee. Neither one means "people who do not work for FBC": a person who works for FBC **and** another company appears in (1).

> **2023/24 Q4(c)(i) (3 × 10m).** S(sid, sname, rating, age), B(bid, bname, colour), R(sid, bid, date).

1. Colours of boats reserved by Albert.

```
SELECT DISTINCT B.colour
FROM S JOIN R ON S.sid = R.sid
       JOIN B ON R.bid = B.bid
WHERE S.sname = 'Albert';
```

> **RA:** π colour (σ sname='Albert' (S) ⋈ R ⋈ B)

2. IDs of sailors who have a rating of at least 8 or reserved boat 103.

- "Or" across two different tables → **union** of two ID sets.

```
SELECT sid FROM S WHERE rating >= 8
UNION
SELECT sid FROM R WHERE bid = 103;
```

> **RA:** π sid (σ rating≥8 (S)) ∪ π sid (σ bid=103 (R))

3. Names of sailors who have not reserved a red boat.

- "Not" → all sailor IDs minus IDs of sailors who reserved a red boat, then join back for names.

```
SELECT sname
FROM S
WHERE sid NOT IN (SELECT R.sid FROM R JOIN B ON R.bid = B.bid
                  WHERE B.colour = 'red');
```

> **RA:** π sname ((π sid (S) − π sid (σ colour='red' (B) ⋈ R)) ⋈ S)

> **2023/24 Q4(c)(ii) (2 × 10m).** Describe what each expression returns, and discuss similarities and differences.

- 1. π sid (σ age>20 (S)) − π sid (σ colour='red' (B) ⋈ R)
- 2. π S.sname [(σ R.sid=R2.sid ∧ R.bid≠R2.bid (R × ρ R2 (R))) ⋈ S]

Expression 1: IDs of sailors **older than 20 who have not reserved any red boat** (sailors over 20, minus sailors who reserved a red boat).

Expression 2: names of sailors who have reserved at least two different boats. R × ρ R2 (R) makes every pair of reservations; σ keeps pairs by the **same sailor** for **different boats**; joining S adds the sailor's name.

Similarities: both are about sailors and their reservations (both use R); both end with a projection that returns sailor identifiers or names.

Differences: (1) uses **set difference** ("not") and returns **sids**; (2) uses **rename + Cartesian product (a self-join)** with ≠ ("at least two") and returns **names**. (1) uses the Boats table (colour); (2) does not. (1) can include sailors with **no** reservations at all; (2) needs **two or more**.

Questions NOT in the mid scope

Past paper questions on Object-Relational databases, Data Warehousing/OLAP, the three-schema architecture and NoSQL are left out, because the mid covers Lessons 1–5 only.